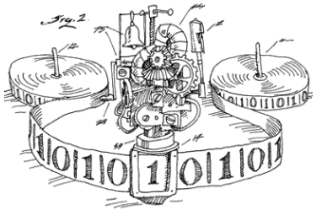
Teoría de la Computación y Lenguajes Formales



Teoría de la Computación y Lenguajes Formales: Sesión 9. Autómatas de Pila (PDA) y Lenguajes Libres de Contexto

Édgar Chacón Ramírez
Departamento de Computación
Semestre 2026 - B

Alcance de la sesión 9

UNIVERSIDAD DE LOS ANDES VENEZUELA

Introducción y Repaso

Pipeline: De la Especificación a la Ejecución

La Memoria Estructurada (Pila LIFO)

Definición Formal del PDA

Operaciones de Pila y Transiciones

Criterios de Aceptación

Representación Gráfica del Autómata de Pila

Representación Matricial

Ejemplo en Workflows: Anidamiento Balanceado

Conclusión

¿Dónde quedamos en la sesión anterior?

UNIVERSIDAD DE LOS ANDES VENEZUELA

Balance de la Sesión 8

- Demostramos mediante el **Lema del Bombeo** que los Autómatas Finitos (DFA) tienen memoria estrictamente finita.
- Un DFA no puede contar de forma ilimitada ni verificar estructuras anidadas o balanceadas ($L = \{a^n b^n\}$).
- Concluimos que para modelar flujos de trabajo reales con **sub-procesos recursivos y holones**, requerimos un autómata con una memoria estructurada: la **Pila (Stack)**.

El Doble Rol de la Pila: En el Parser y en la Planta

UNIVERSIDAD DE LOS ANDES VENEZUELA

1. Rol Sintáctico (Al leer el archivo YAML / JSON)

El parser que lee nuestro archivo de configuración utiliza internamente un **Autómata de Pila (PDA)**:

- Al encontrar un nuevo nivel de sangría/bloque anidado $\rightarrow$ hace **PUSH** del nivel de espacios.
- Al reducir la sangría $\rightarrow$ hace **POP** para cerrar el bloque contextual.
- Un simple DFA sería incapaz de validar la sintaxis de indentación de YAML.

2. Rol Operacional (Al ejecutar el workflow en la planta)

El motor de ejecución utiliza la **Pila de Procesos**:

- Al invocar un sub-proceso subordinado $\rightarrow$ hace **PUSH** del estado del proceso padre.
- Al terminar el sub-proceso $\rightarrow$ hace **POP** para recuperar el contexto y continuar el flujo.

Arquitectura: Del Archivo al Motor en Ejecución

UNIVERSIDAD DE LOS ANDES VENEZUELA

El Pipeline de Procesamiento

Para que un flujo de trabajo se ejecute automáticamente en el sistema, debe transitar por cuatro niveles estrictos:

1. **Descripción Textual (YAML / BNF):** El documento que define el proceso y los recursos.
2. **Validación Sintáctica (PDA Sintáctico):** ¿Cumple la gramática del metalenguaje (indentación, estructura)?
3. **Validación Ontológica / Semántica:** ¿Cumple las reglas del dominio (recursos, estados válidos, jerarquías permitidas)?
4. **Modelo en Memoria:** Construcción del objeto formal (5-tupla o 7-tupla) en la RAM.
5. **Motor en Ejecución (PDA Operacional / DES):** El intérprete evaluando eventos en tiempo real.

Validación Sintáctica vs. Validación Ontológica

UNIVERSIDAD DE LOS ANDES VENEZUELA

Dos niveles de verificación indispensables antes de instanciar el modelo

Nivel	¿Qué valida?	Ejemplo de Error Detectado
Sintáctico *(Forma / Gramática)*	Reglas del formato de escritura (YAML, JSON, BNF). El parser no sabe qué es una fábrica.	Error de indentación en YAML, falta de dos puntos :, tipos de datos incorrectos (string en lugar de lista).
Ontológico *(Semántica / Dominio)*	Coherencia con el modelo de negocio, la física del recurso y el catálogo de conceptos.	Transición hacia un estado inexistente ($q \notin Q$), estados finales no contenidos en Q ($F \not\subseteq Q$), o asignar operaciones incompatibles con el tipo de recurso.

Principio Fundamental

Un archivo puede ser **sintácticamente perfecto** (YAML válido) y ser **ontológicamente erróneo** (un proceso absurdo o irrealizable). El sistema debe validar ambos niveles.

Superando la Memoria Finita: La Estructura de Pila (LIFO)

¿Por qué añadir una Pila al Autómata?

UNIVERSIDAD DE LOS ANDES VENEZUELA

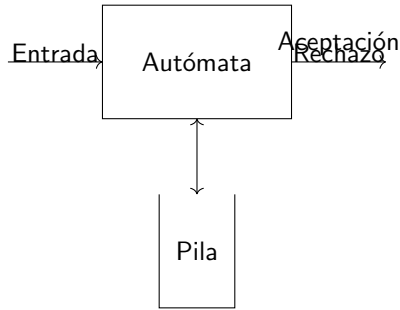
Recordatorio de la Sesión 8

Demostramos mediante el Lema del Bombeo que un DFA no puede reconocer lenguajes como $L = \{a^n b^n\}$, porque carece de memoria auxiliar para contar o recordar estructuras anidadas.

La Pila (Stack)

Para dotar al autómata de memoria auxiliar sin llegar a la complejidad total de una Máquina de Turing, añadimos un almacenamiento con acceso restringido:

- Funciona bajo el principio **LIFO** (*Last-In, First-Out*: el último en entrar es el primero en salir).
- Solo se puede consultar o modificar el elemento en el **tope de la pila**.
- Capacidad no acotada: permite almacenar una cantidad arbitraria de símbolos.



La Pila en la Programación y en los Workflows

UNIVERSIDAD DE LOS ANDES VENEZUELA

Analogía 1: La Pila de Ejecución (*Call Stack*)

En cualquier lenguaje de programación, cuando una función $f()$ llama a $g()$ y esta a $h()$, las direcciones de retorno se apilan. Al terminar $h()$, se desapila para volver exactamente a $g()$.

Analogía 2: Procesos Holónicos Jerárquicos

En un flujo de trabajo industrial:

- Iniciar un sub-proceso $\rightarrow$ **Apilar (PUSH)** el estado del proceso padre.
- Ejecutar tareas del sub-proceso $\rightarrow$ Trabajar en el nivel actual.
- Concluir sub-proceso $\rightarrow$ **Desapilar (POP)** y reanudar el proceso contenedor.

La Definición Matemática: La 7-Tupla del Autómata de Pila

La 7-Tupla Formal

UNIVERSIDAD DE LOS ANDES VENEZUELA

Definición de Autómata de Pila (PDA)

Un Autómata de Pila se define formalmente como una 7-tupla:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Componentes del Formalismo

Símbolo	**Significado Clásico**	**Equivalente en Workflow**
Q	Conjunto finito de estados de control	Estados del coordinador / recurso
Σ	Alfabeto de entrada	Eventos del proceso
Γ	Alfabeto de la pila	Tipos de tareas/sub-holones en espera
δ	Función de transición extendida	Reglas de flujo con gestión de contexto
$q_0 \in Q$	Estado inicial	Estado de partida
$Z_0 \in \Gamma$	Símbolo inicial de fondo de pila	Marcador de base / pila limpia
$F \subseteq Q$	Conjunto de estados de aceptación	Estados de éxito / terminación global

La Función de Transición δ

UNIVERSIDAD DE LOS ANDES VENEZUELA

Firma de la Función

La función de transición mapea el estado actual, el símbolo leído en la cinta (o ε) y el símbolo en el tope de la pila hacia un nuevo estado y una cadena de reemplazo en la pila:

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to \mathcal{P}(Q \times \Gamma^*)$$

Interpretación de $\delta(q, a, Z) = \{(p, \gamma)\}$

Estando en el estado q, leyendo el evento a de la cinta y teniendo el símbolo Z en el tope de la pila:

1. El autómata cambia al estado p.
2. El símbolo Z se retira del tope y se sustituye por la cadena $\gamma \in \Gamma^*$.

Operaciones Básicas: PUSH, POP y Transiciones ε

Mecánica de las Operaciones sobre la Pila

UNIVERSIDAD DE LOS ANDES VENEZUELA

Los tres casos según la cadena de reemplazo γ:

- **POP (Desapilar):** Si $\gamma = \varepsilon$, el símbolo Z se elimina del tope.

$$\delta(q, a, Z) = (p, \varepsilon)$$

- **NO-OP / Cambio de símbolo:** Si $\gamma = Z$, el tope no cambia. Si $\gamma = Y$, Z se reemplaza por Y.

$$\delta(q, a, Z) = (p, Z) \quad \text{o} \quad \delta(q, a, Z) = (p, Y)$$

- **PUSH (Apilar):** Si $\gamma = YZ$, se coloca Y encima de Z (la pila crece).

$$\delta(q, a, Z) = (p, YZ)$$

Transiciones ε (Espontáneas)

Si la entrada leída es ε, el autómata modifica la pila y cambia de estado **sin consumir ningún símbolo de la cinta**.

Criterios de Aceptación: Estado Final vs. Pila Vacía

Formas de Aceptar una Cadena

UNIVERSIDAD DE LOS ANDES VENEZUELA

1. Aceptación por Estado Final

La cadena w es aceptada si al consumirla por completo, el autómata llega a algún estado de aceptación $q \in F$, sin importar qué quede en la pila:

$$L(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \gamma), q \in F\}$$

2. Aceptación por Pila Vacía

La cadena w es aceptada si al consumirla por completo, la pila queda totalmente vacía, sin importar en qué estado quede el autómata:

$$N(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (q, \varepsilon, \varepsilon)\}$$

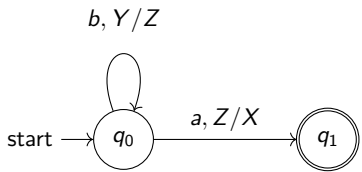
Teorema de Equivalencia

Ambos criterios son computacionalmente **equivalentes**: para todo PDA que acepta por estado final, existe un PDA equivalente que acepta por pila vacía, y viceversa.

Representación Gráfica del Autómata de Pila

Formato de las etiquetas de transición

A diferencia de los diagramas convencionales (AFD/AFnD), cada transición en un grafo de un PDA se etiqueta bajo la convención: **Entrada, Tope / Reemplazo**.



- **Lazo en q_0 $(b, Y/Z)$:** Lee el símbolo b de la cinta, requiere que en el tope de la pila esté Y, y lo reemplaza por Z (un cambio de símbolo).
- **Transición a q_1 $(a, Z/X)$:** Lee el símbolo a, verifica el tope Z, y lo reemplaza por X.

Representación Matricial del Autómata a Pila

UNIVERSIDAD DE LOS ANDES VENEZUELA

Matriz Bidimensional de Doble Entrada

La función de transición δ se organiza mapeando el **Estado Actual** contra pares compuestos por el **(Símbolo leído, Tope de pila)**. Las celdas indican el **(Siguiente Estado, Cadena a Apilar)**.

Estado	**(a, Z_0)**	**(a, X)**	**(b, X)**	**(ε, Z_0)**
q_0	(q_0, XZ_0)	(q_0, XX)	(q_1, ε)	(q_2, Z_0)
q_1	$\emptyset$	$\emptyset$	(q_1, ε)	(q_2, Z_0)
q_2	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

- Las celdas marcadas con $\emptyset$ representan transiciones no definidas o estados de rechazo (Error).

UNIVERSIDAD DE LOS ANDES VENEZUELA

Caso Práctico: Validación de Sub-holones Balanceados

El PDA para el Lenguaje $L = \{a^n b^n \mid n \geq 1\}$

UNIVERSIDAD DE LOS ANDES VENEZUELA

Definición de la 7-Tupla

- $Q = \{q_0, q_1, q_2\}, \quad \Sigma = \{a, b\}, \quad \Gamma = \{A, Z_0\}, \quad F = \{q_2\}$

Reglas de Transición:

1. $\delta(q_0, a, Z_0) = (q_0, AZ_0)$ *(Primer evento a: apila A sobre Z_0)*
2. $\delta(q_0, a, A) = (q_0, AA)$ *(Siguientes eventos a: apila A sobre A)*
3. $\delta(q_0, b, A) = (q_1, \varepsilon)$ *(Primer evento b: pasa a q_1 y desapila una A)*
4. $\delta(q_1, b, A) = (q_1, \varepsilon)$ *(Siguientes eventos b: desapila una A por cada b)*
5. $\delta(q_1, \varepsilon, Z_0) = (q_2, Z_0)$ *(Pila en base Z_0: pasa al estado de aceptación q_2)*

Traza de Ejecución para $w = aabb$

UNIVERSIDAD DE LOS ANDES VENEZUELA

Evolución de las Configuraciones Instantáneas: (q, w, α)

$$
\begin{aligned}
(q_0, aabb, Z_0) &\vdash (q_0, abb, AZ_0) && \text{[Regla 1: Lee } a\text{, PUSH } A\text{]} \\
&\vdash (q_0, bb, AAZ_0) && \text{[Regla 2: Lee } a\text{, PUSH } A\text{]} \\
&\vdash (q_1, b, AZ_0) && \text{[Regla 3: Lee } b\text{, POP } A\text{]} \\
&\vdash (q_1, \varepsilon, Z_0) && \text{[Regla 4: Lee } b\text{, POP } A\text{]} \\
&\vdash (q_2, \varepsilon, Z_0) && \text{[Regla 5: Transición } \varepsilon\text{, Éxito en } q_2 \in F\text{]}
\end{aligned}
$$

Interpretación en Producción

$a =$ abrir_fase_de_ensamble, $b =$ cerrar_fase_de_ensamble.

La pila garantiza que toda fase abierta se cierre en orden inverso estricto antes de declarar el producto completado.

Síntesis: ¿Cómo la Teoría Formal Resuelve el Problema?

UNIVERSIDAD DE LOS ANDES VENEZUELA

El Mapa de Herramientas Formales en Nuestra Arquitectura

Formalismo	**¿Qué problema resuelve en el software?**	**¿Qué problema resuelve en la planta?**
DFA (Tipo 3)	Validador de expresiones regulares y cadenas simples de eventos.	Ciclo de vida y estados operativos de un recurso individual sin sub-procesos.
PDA (Tipo 2)	**Parser sintáctico de YAML/JSON** con control de sangrías y bloques anidados.	**Motor de sub-workflows recursivos** con gestión de contexto (PUSH/POP de holones).
Redes de Petri *(Próximo Bloque)*	Coordinador concurrente de eventos asíncronos y paso de mensajes.	Sincronización de múltiples recursos cooperantes e intercambio del producto físico.

Conclusión de la Sesión 9

UNIVERSIDAD DE LOS ANDES VENEZUELA

Lo que aprendimos hoy

- **La Pila como Memoria Dinámica:** Permite manejar conteos ilimitados y estructuras jerárquicas sin perder el control formal.
- **La 7-Tupla del PDA:** Formaliza la transición considerando simultáneamente el estado, la entrada y el tope de la pila.
- **Operaciones LIFO:** PUSH para entrar en contextos anidados, POP para retornar y validar balances.
- **Doble Aplicación:** La pila es el motor del parser sintáctico de archivos y el administrador de sub-procesos en ejecución.

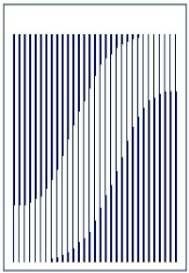
Próxima sesión (Hito 2)

Extenderemos nuestro motor en Python (`recurso.py`) incorporando la pila física para interpretar archivos YAML con sub-procesos recursivos.

¡Muchas gracias!



UNIVERSIDAD
DE LOS ANDES
VENEZUELA



Édgar Chacón Ramírez
email: echacon@ula.ve